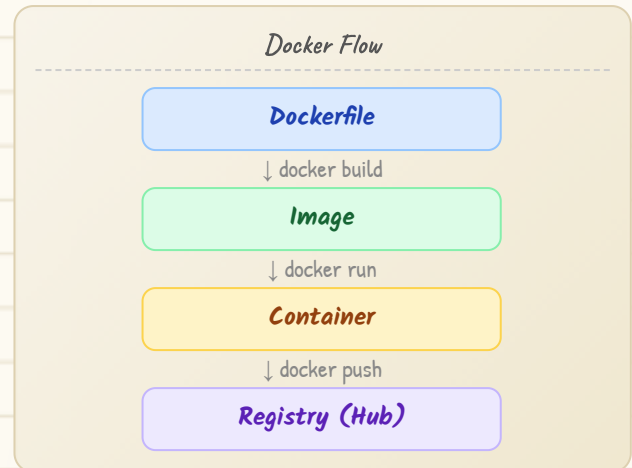


What is Docker?

1 What is Docker?

- **Docker** is an open-source **containerization** platform.
- Packages apps into lightweight, portable **containers**.
- Containers share the host OS kernel -- **no full VM needed**.
- Ensures "**works on my machine**" actually works everywhere.
- Built on Linux namespaces, cgroups and UnionFS.



2 Containers vs Virtual Machines

Containers	Virtual Machines
Share host OS kernel	Each has full guest OS
Lightweight (MBs)	Heavy (GBs)
Start in seconds	Start in minutes
Less isolation (process level)	Full isolation (hardware level)

★ Remember

- ✓ Docker containers are **NOT** VMs -- they share the host kernel
- ✓ Image = blueprint, Container = running instance of an image

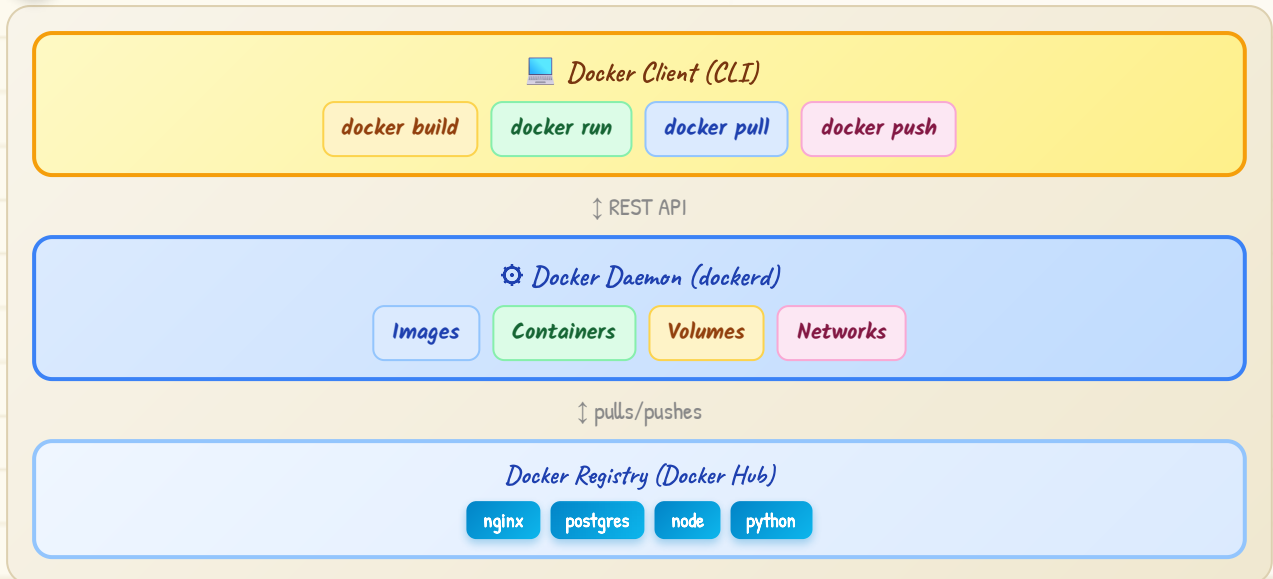
💡 Interview Tip

Q: What is Docker?

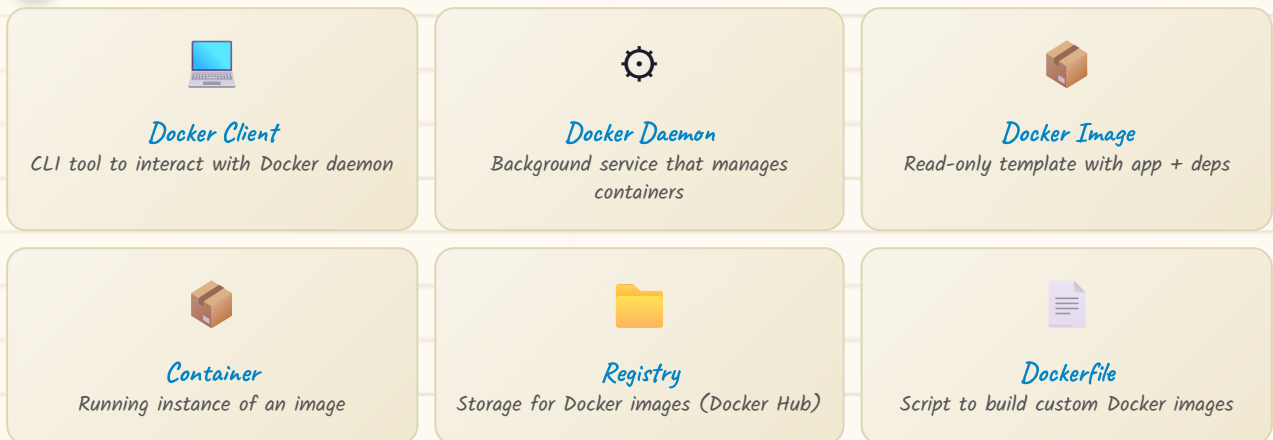
A: Docker is an open-source platform that automates the deployment of applications inside lightweight, portable containers that share the host OS kernel.

Docker Architecture

1 Client-Server Architecture



2 Key Components



💡 Interview Tip

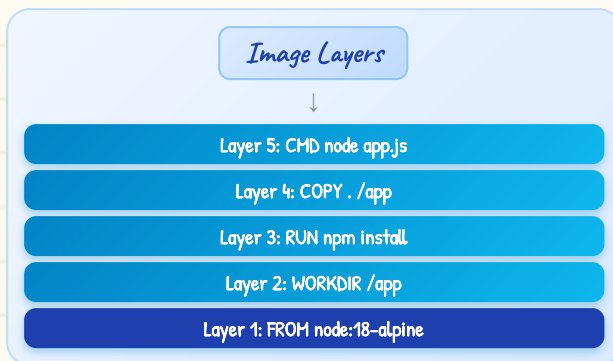
Q: Explain Docker architecture.

A: Docker uses a client-server model. The Docker Client sends commands via REST API to the Docker Daemon, which builds, runs, and manages containers. Images are stored in registries like Docker Hub.

Images & Containers

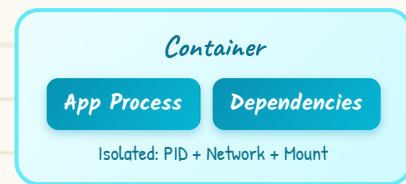
1 Docker Image

- **Read-only** template with app code + dependencies
- Built from a **Dockerfile** using layers
- Each instruction = one **layer** (cached!)
- Stored in registries (Docker Hub, ECR, GCR)
- Tagged as `name:tag` (e.g., `node:18-alpine`)

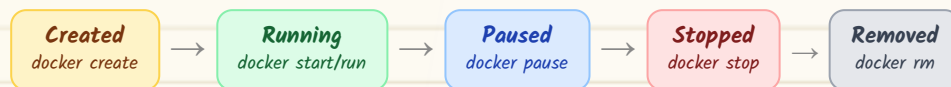


2 Docker Container

- **Running instance** of a Docker image
- Has its own **filesystem, network, processes**
- Writable layer on top of read-only image layers
- Isolated via **namespaces + cgroups**
- Ephemeral -- data lost when removed (use volumes!)



3 Container Lifecycle



💡 Interview Tip

Q: Difference between Image and Container?

A: An Image is a read-only template (like a class). A Container is a running instance of that image (like an object). Images have layers; containers add a writable layer on top.

Dockerfile & Best Practices

1 Key Dockerfile Instructions

Instruction	Purpose	Example
<i>FROM</i>	Base image to start from	FROM node:18-alpine
<i>WORKDIR</i>	Set working directory	WORKDIR /app
<i>COPY</i>	Copy files from host to image	COPY package.json .
<i>RUN</i>	Execute command during build	RUN npm install
<i>EXPOSE</i>	Document listening port	EXPOSE 3000
<i>CMD</i>	Default command at runtime	CMD ["node", "app.js"]
<i>ENV</i>	Set environment variables	ENV NODE_ENV=prod

2 Sample Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

3 Best Practices

- Use **alpine** base images (smaller)
- **Multi-stage builds** to reduce image size
- Copy **package.json** first for layer caching
- Use **.dockerignore** to exclude files
- Don't run as **root** user
- Combine RUN commands with **&&**

★ CMD vs ENTRYPOINT

- ✓ CMD -- default command, can be overridden at runtime
- ✓ ENTRYPOINT -- always runs, CMD args appended to it
- ✓ Use together: ENTRYPOINT ["python"] + CMD ["app.py"]

Volumes & Networking

1 Docker Volumes

- **Persist data** beyond container lifecycle
- Managed by Docker, stored on host filesystem
- Shared between multiple containers
- Better than bind mounts for production

Volumes
Docker-managed,
best for
persistent data

Bind Mounts
Maps host path
into container

tmpfs
In-memory only,
lost on restart

```
$ docker volume create mydata
$ docker run -v mydata:/app/data myimg
```

2 Docker Networking

- Containers communicate via **Docker networks**
- DNS-based service discovery by container name
- Port mapping with **-p host:container**

Driver	Use Case
bridge	Default. Containers on same host talk to each other
host	Container uses host network directly
none	No networking, fully isolated
overlay	Multi-host networking (Docker Swarm)

★ Key Points

- ✓ Always use **named volumes** for databases
- ✓ Use **custom bridge networks** for DNS discovery
- ✓ **-p 8080:3000** maps host 8080 to container 3000

💡 Interview Tip

Q: Volume vs Bind Mount?

A: Volumes are Docker-managed and portable. Bind mounts depend on host file structure. Volumes are preferred for production; bind mounts for development.

Docker Compose & Quick Reference

1 Docker Compose

- Define **multi-container** apps in one YAML file
- One command to start **all services**
- Auto-creates a shared **network**
- Great for **local dev** environments

```
version: "3.8"
services:
  web:
    build: .
    ports: ["3000:3000"]
    depends_on: [db]
  db:
    image: postgres:15
    volumes:
      [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

★ Docker Cheat Sheet

- ✓ Image -- read-only template (layers)
- ✓ Container -- running instance of image
- ✓ Volume -- persistent data storage
- ✓ Network -- container communication
- ✓ Dockerfile -- script to build images
- ✓ Compose -- multi-container orchestration

2 docker CLI Quick Reference

```
docker build -t myapp .
```

Build image from Dockerfile

```
docker run -d -p 80:3000 myapp
```

Run container in background

```
docker ps / docker ps -a
```

List running / all containers

```
docker logs <container>
```

View container logs

```
docker exec -it <c> /bin/sh
```

Shell into running container

```
docker compose up -d
```

Start all Compose services

```
docker image prune -a
```

Remove all unused images

💡 Final Tip

Practice: Containerize a simple app with Dockerfile, add a database with Docker Compose. Hands-on beats theory!

```
docker init
```

 scaffolds Dockerfile + Compose for you.